ServiX

Arquitectura y diseño del software

Estructura IEEE 1016-2009

Campo	Detalle
Proyecto	ServiX: aplicación Android para gestión de servicios técnicos
Asignatura	Optativa II · Gestión de proyectos integradores con Scrum
Equipo	Equipo ServiX (Juan Esteban Olaya Muñoz, Nicolas Quiroz Gomez, Juan Sebastian Garzon Suarez)
Versión / fecha	1.0 · 01/11/2026

Tabla de contenido

Tabla de contenido 2

1. Introducción 3

1.1 Propósito y alcance . . . 3

1.2 Referencias . . . 3

2. Partes interesadas y preocupaciones 3

3. Puntos de vista de diseño 3

3.1 Contexto . . . 3

3.2 Composición: capas y paquetes . . . 4

3.3 Componentes . . . 5

3.4 Lógico: clases de dominio . . . 5

3.5 Información: modelo de datos . . . 6

3.6 Interacción . . . 6

3.7 Casos de uso . . . 8

3.8 Despliegue . . . 8

4. Decisiones de diseño 9

5. Seguridad 10

6. Limitaciones conocidas 10

1. Introducción

1.1 Propósito y alcance

Describe el diseño de ServiX siguiendo la estructura de IEEE 1016-2009 (Software Design Descriptions): partes interesadas, puntos de vista y decisiones de diseño. Se verificó contra el código fuente y `docs/ARQUITECTURA.md`.

Aclaración sobre las normas. Este documento usa la estructura y las prácticas de las normas citadas como guía de organización. No constituye una certificación ni una declaración de conformidad con IEEE o ISO.

1.2 Referencias

- IEEE Std 1016-2009, Software Design Descriptions.
- ISO/IEC/IEEE 42010:2022, Architecture description (vocabulario de vistas y puntos de vista).
- SRS de ServiX (`SRS_Requisitos`).

2. Partes interesadas y preocupaciones

Parte interesada	Preocupaciones
Equipo de desarrollo	Mantenibilidad, pruebas, integración continua.
Product Owner	Funcionalidades entregadas y trazables.
Administrador / Vendedor	Gestión eficiente de clientes y servicios.
Cliente	Solicitar servicios y recibir atención.
Docente evaluador	Cumplimiento de la guía de Azure DevOps y calidad de la documentación.

3. Puntos de vista de diseño

3.1 Contexto

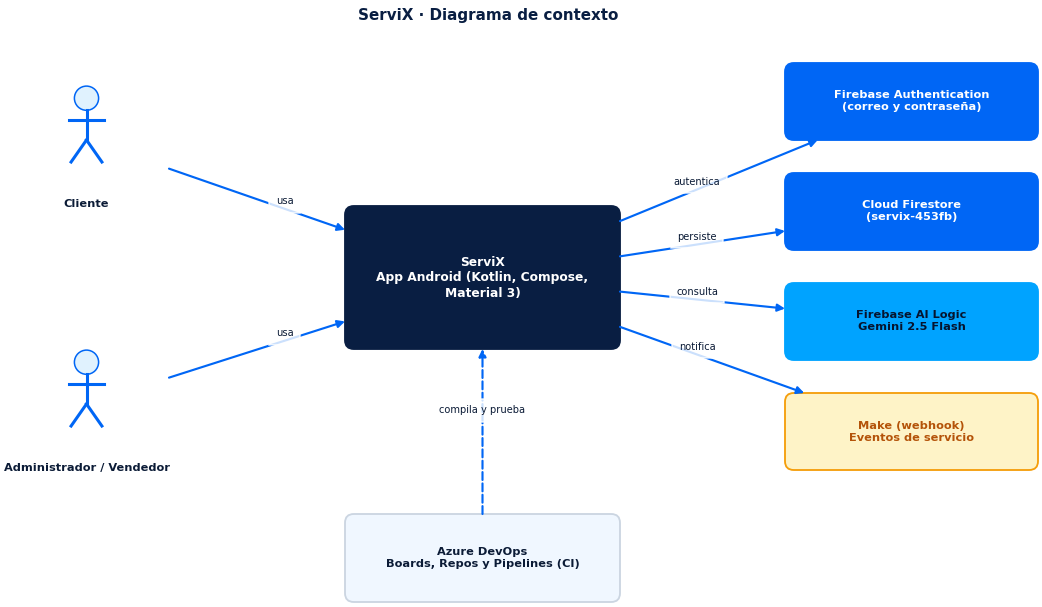


Figura 1. Contexto del sistema.

3.2 Composición: capas y paquetes

La aplicación se organiza en cuatro capas con dependencias dirigidas hacia el dominio. Flujo típico: `Screen` → `MainViewModel` → `UseCase` → `Repository (interfaz)` → `implementación Firebase/OkHttp`.

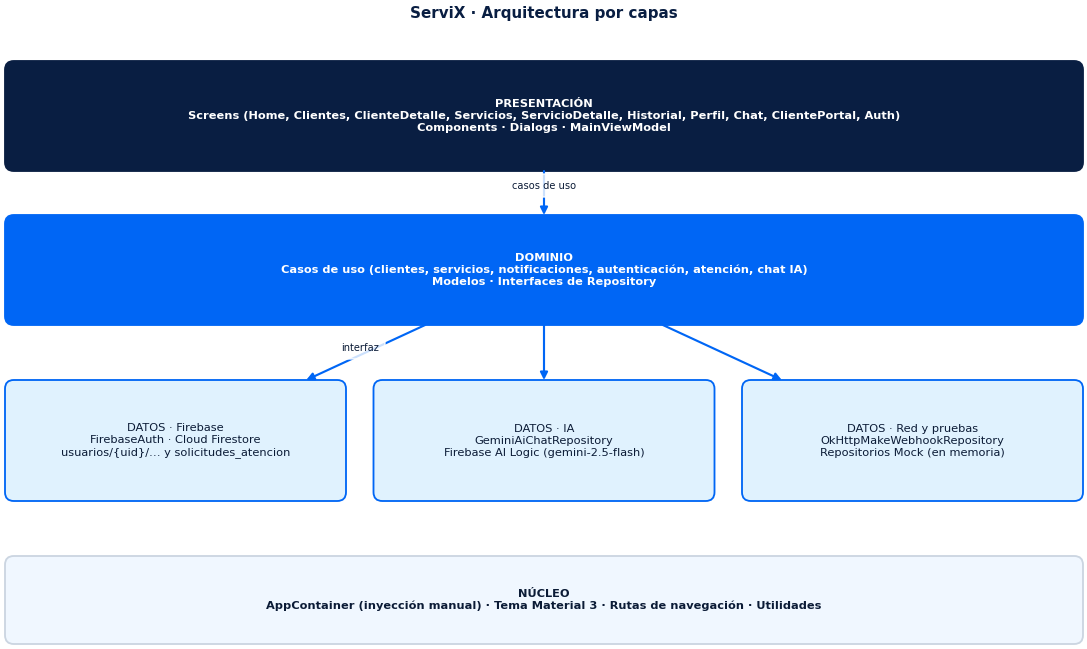


Figura 2. Arquitectura por capas.

Paquete	Contenido	Responsabilidad
`presentation/screens`	Home, Clientes, ClienteDetalle, Servicios, ServicioDetalle, Historial, Perfil, Chat, ClientePortal	Pantallas Compose
`presentation/components`, `dialogs`	Tarjetas, chips, stepper, formularios, búsqueda global, centro de notificaciones	UI reutilizable
`presentation/viewmodel`	`MainViewModel`, `MainViewModelFactory`	Estado de UI y orquestación
`ui/screens`	`AuthScreen`	Inicio de sesión, registro y recuperación
`domain/model`	Cliente, Servicio, Notificacion, Actividad, SolicitudAtencion, UserProfile, UserRole…	Entidades de negocio
`domain/usecase`	Casos de uso de clientes, servicios, notificaciones, autenticación, atención y chat IA	Reglas de negocio
`domain/repository`	Interfaces de acceso a datos	Contratos
`data/firebase`, `data/ai`, `data/network`, `data/mock`	Implementaciones	Persistencia, IA, webhook y dobles de prueba
`core`	`AppContainer`, tema, rutas, utilidades	Inyección manual y soporte

3.3 Componentes

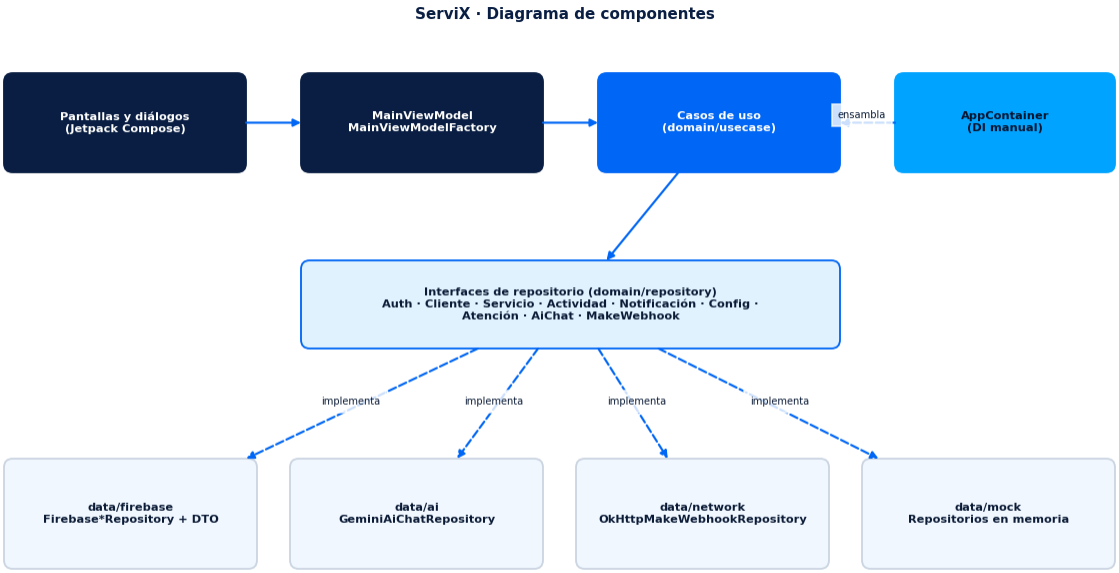


Figura 3. Componentes y dependencias.

3.4 Lógico: clases de dominio

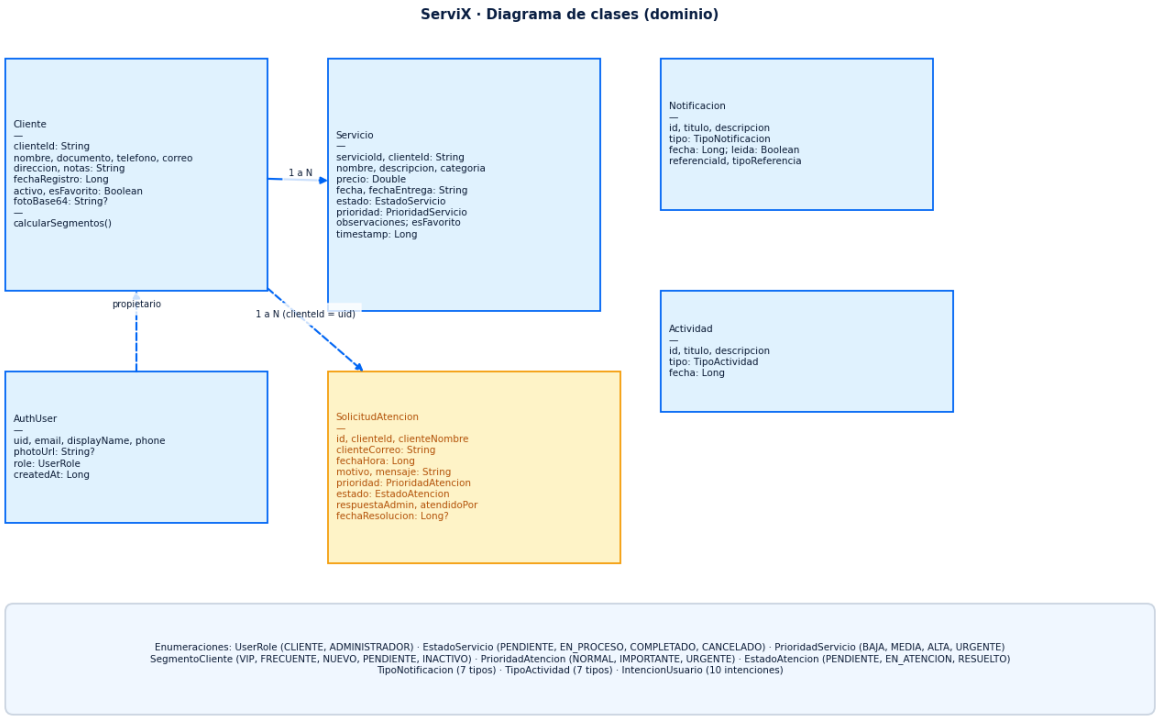


Figura 4. Clases de dominio y enumeraciones.

3.5 Información: modelo de datos

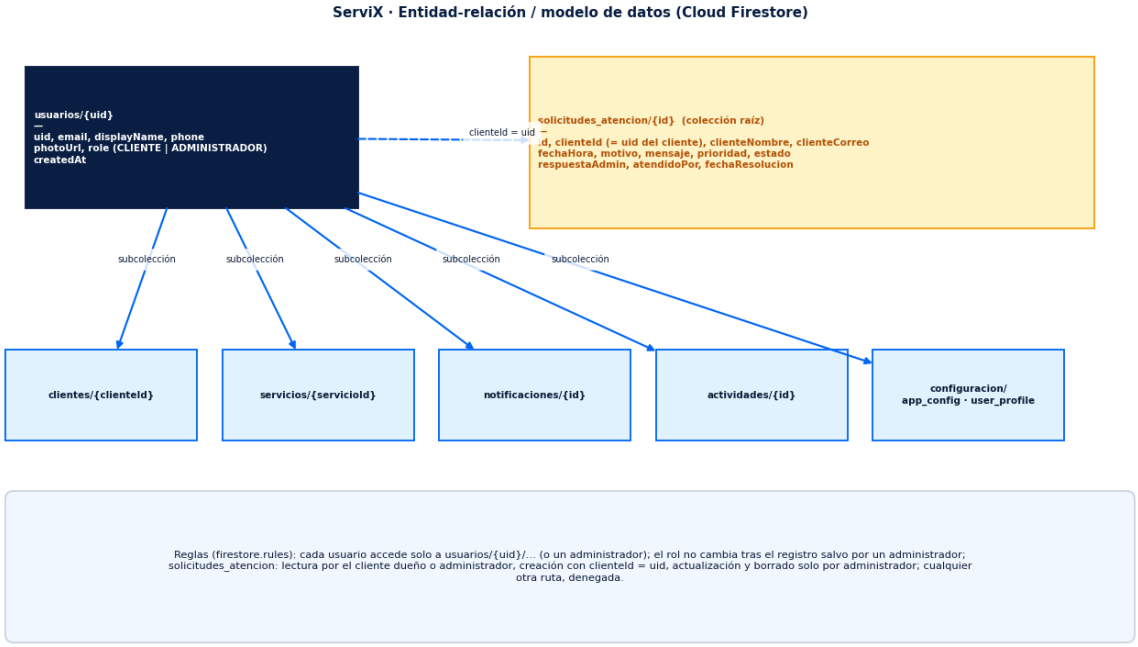


Figura 5. Modelo de datos en Firestore.

3.6 Interacción

Los siguientes diagramas de secuencia describen los tres flujos principales.



Figura 6. Inicio de sesión.

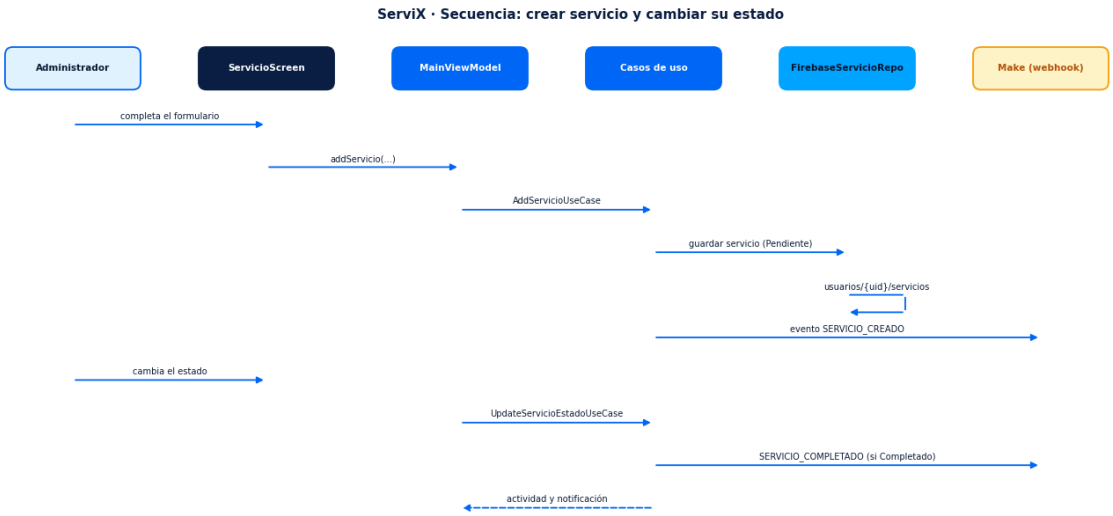


Figura 7. Crear un servicio y cambiar su estado.



Figura 8. Chat con IA y escalamiento.

3.7 Casos de uso

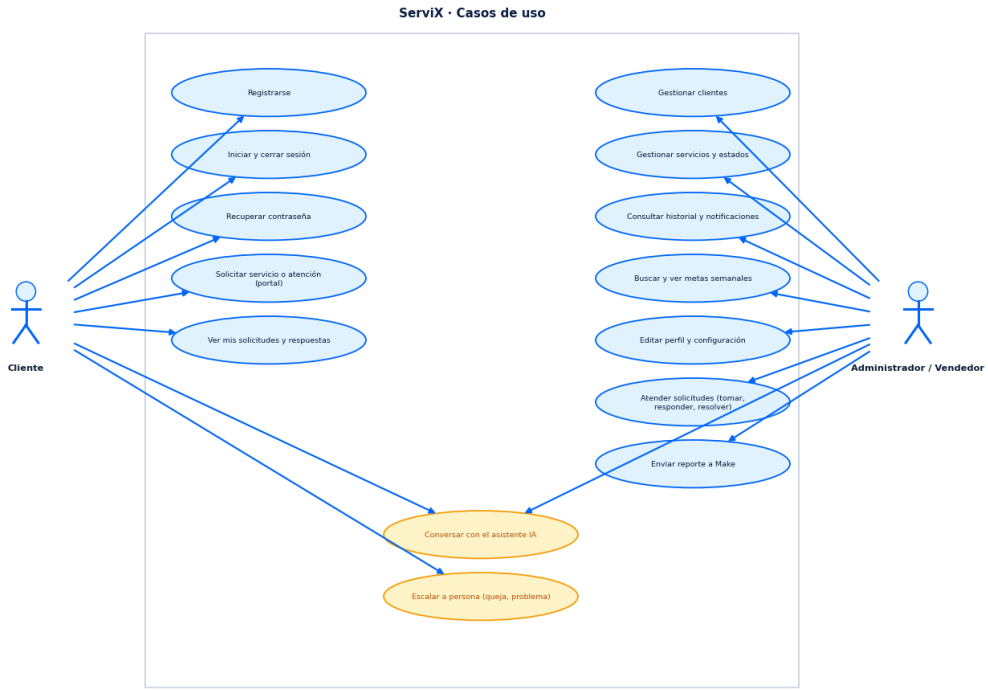


Figura 9. Casos de uso por actor.

3.8 Despliegue

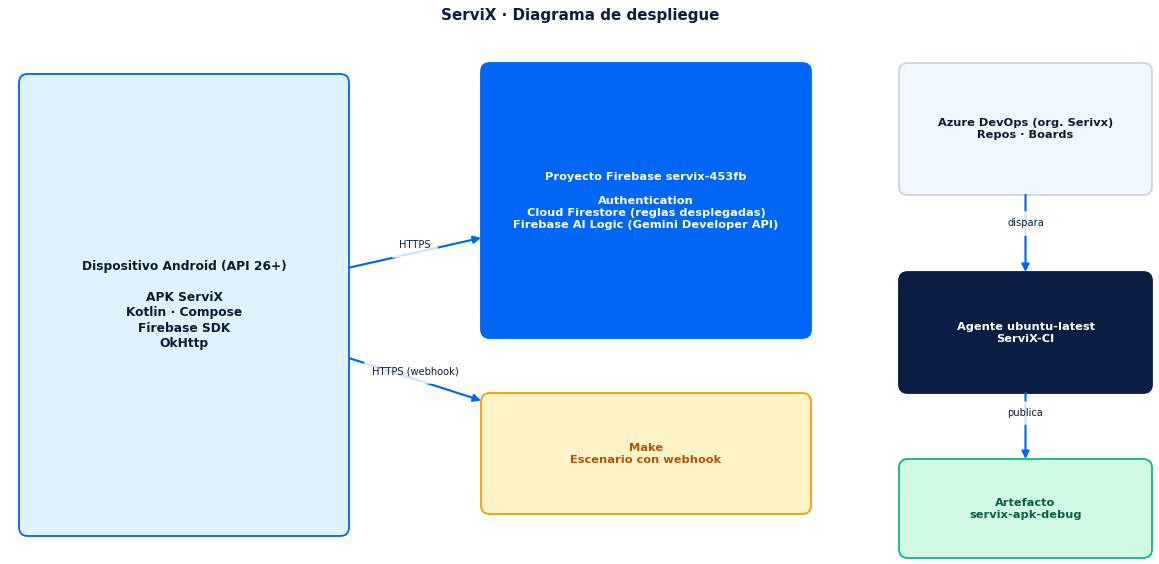


Figura 10. Despliegue.

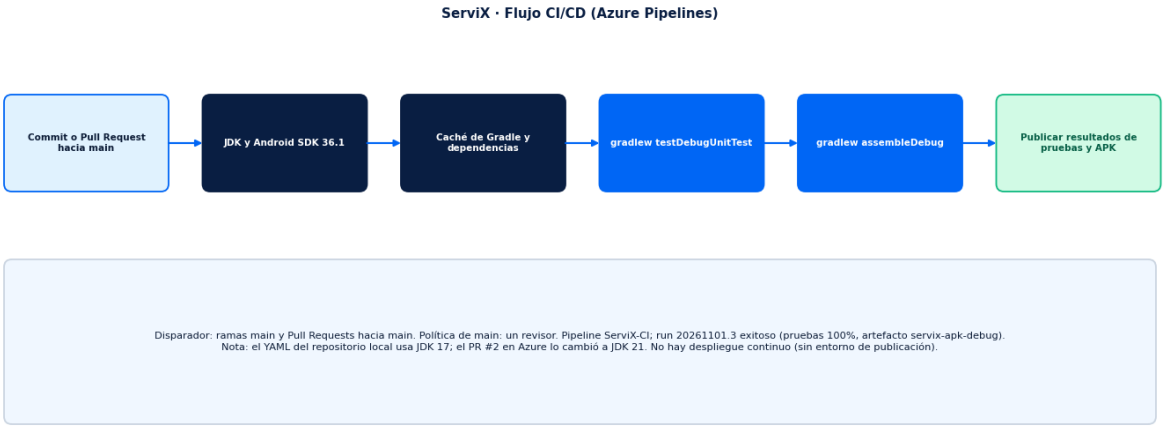


Figura 11. Flujo de CI/CD en Azure Pipelines.

4. Decisiones de diseño

Decisión	Motivo	Consecuencia
Arquitectura por capas con casos de uso	Separar UI, reglas y datos.	Se pueden sustituir repositorios por dobles en pruebas.
DI manual con `AppContainer` (sin Hilt ni Koin)	Simplicidad y dependencias mínimas.	Los repositorios son singletons perezosos; si Firebase falla se degrada a mocks sin aviso (H-11).
Firestore con árbol por usuario `usuarios/{uid}/…`	Aislamiento de datos y reglas simples.	Las reglas se basan en pertenencia y rol.
Firebase AI Logic en lugar de clave en el cliente	Evitar exponer `GEMINI_API_KEY` en el APK (H-02).	La credencial la gestiona el proyecto Firebase; se recomienda App Check.

Decisión	Motivo	Consecuencia
Clasificación de intención por palabras clave	Escalar de forma determinista casos sensibles.	Cobertura limitada del lenguaje natural.
Webhook de Make con OkHttp	Automatización rápida sin backend propio.	La URL está en el código (H-05); debería moverse a un backend.

5. Seguridad

- Reglas de Firestore con mínimo privilegio, desplegadas en `servix-453fb`.
- El rol se lee de la cuenta almacenada; si no coincide con el elegido se rechaza el acceso (H-18); si falla la lectura se asume `CLIENTE` (H-17).
- Atajos de prueba solo en builds debug (H-04).
- Sin claves de IA en el cliente (H-02).

6. Limitaciones conocidas

- `MainViewModel` concentra gran parte del estado (cerca de 25 KB) y es candidato a dividirse.
- `AppContainer` degrada a repositorios simulados sin informar al usuario (H-11).
- `SignUpUseCase` y `AuthUser` usan `ADMINISTRADOR` como valor por defecto, lo que contradice el mínimo privilegio aplicado en H-17; se recomienda revisarlo.
- Las interfaces `RetellAIRepository` y `N8nWebhookRepository` no tienen implementación (H-13).
- Los diagramas PNG se generaron con un script propio y son una vista aproximada de los archivos `.drawio`, que son la fuente editable.